2025 - October - Data Security

Jameel AHMED - University of Law
Created: 08-DEC-25 **Last Modified:** 11-JAN-26

PebblePad

Contents

Cover Sheet

Task 1

Task 2

Task 3

Task 4

References

Cover Sheet

FRONT SHEET

Individual Coursework

CANDIDATE NUMBER (C-NUMBER)

415130

MODULE NAME

Data Security

WORD COUNT

2250

SUBMISSION DATE

11-Jan-2026

DECLARATION

I certify that this assessment submission is entirely my work and I have fully referenced and correctly cited the work of others, where required. I also confirm the contents of my submission have not been generated by a third party, or through an Artificial Intelligence generative system*.

I have read the Student Discipline Regulations (Student Discipline Regulations) and understand any Assessment Related Offence/ Academic Misconduct may result penalties being applied.

By submitting this assessment submission, I am confirming that I am fit to sit according to the Assessment Regulations.

I declare that: This is my own unaided work.

Yes

No

The word count stated by me is correct

Yes

No

I’m happy for my work to be retained on the Elite repository and made available to staff and future students**

Yes

No

*Please note that all the assignments are submitted to Turnitin.

**Please note personal information (such as names) will be deleted

Task 1: Critically evaluate and assess the data security risks in the given scenario above.

Overview of Recent Incidents

From the scenario, eSecureFin has experienced three specific security incidents in the past year:

- **Data Breach:** Thousands of client's data exposed due to unauthorized access to company's database.
- **Phishing Attacks:** Fraudulent emails received to employess led them to compromise credentials
- **Insider Threat:** Before leaving the company a former employee steal sensitive data

These incidents measure company's outdated measures and resulting in compromise of CIA triad.

1. Risk Matrix (Likelihood vs Impact)

Likelihood and impact are assessed considering the company's current weak controls and the fact that these incidents have already occurred.

- Likelihood: 1=Rare, 2=Unlikely, 3=Possible, 4=Likely, 5=Almost Certain
- Impact: 1=Insignificant, 2=Minor, 3=Moderate, 4=Major, 5=Catastrophic
- Risk Score = Likelihood × Impact; Levels: Low ≤8, Medium 9–15, High 16–20, Extreme >20

Recent Incident	Likelihood	Impact	Risk Score	Risk Level	Analysis Based on Scenario
Phishing Attacks	4 (Likely)	4 (Major)	16	High	Already successful; weak training and access controls make repeat attacks highly probable, leading to widespread credential loss and follow-on breaches.
Insider Threat	3 (Possible)	5 (Catastrophic)	15	Medium-High	Occurred due to poor off-boarding and monitoring; retained privileges enabled data theft with severe regulatory and reputational consequences.
Data Breach (Database Exposure)	4 (Likely)	5 (Catastrophic)	20	Extreme	Direct unauthorized database access already happened; inadequate encryption and weak controls increase likelihood of recurrence with massive client data exposure.

- **Risk Matrix Summary:** Based on selected because of the simplicity of the graphical representation of risks, in terms of probability and impact.
- **Relates to the scenario:** Assists in prioritizing the three recent events of eSecureFin (phishing attack, insider threat attack, breach of data) depending on the chances of reoccurrence and its impact on the business.
- **Analysis of Matrix:** The data breach is the most critical risk as it exposed thousands of records directly. Phishing is second one acts as enabler for other attacks.

2. CVSS Scoring

Scores reflect the vulnerabilities exploited in recent incidents at eSecureFin.

Recent Incident	CVSS Base Score	Severity	Key Metrics Contributing to Score
Phishing Attacks	7.6 (High)	High	Attack Vector: Network; Attack Complexity: Low; Privileges Required: None; User Interaction: Required; Confidentiality: High (compromised credentials led to system access).
Insider Threat	8.3 (High)	High	Attack Vector: Network; Attack Complexity: Low; Privileges Required: Low (retained access); User Interaction: None; Confidentiality: High; Scope: Unchanged.
Data Breach (Database Exposure)	9.1 (Critical)	Critical	Attack Vector: Network; Attack Complexity: Low; Privileges Required: Low; Confidentiality/Integrity: High; Availability: High (thousands of records exposed due to weak controls).

- **CVSS Summary:** Selected because it has an objective rating scale (from 0 to 10) of the technical vulnerabilities.
- **Links to scenario:** Captures the exploited vulnerabilities in the recent incidents (namely, low complexity and high confidentiality impact) so that comparisons and justifications for immediate patches in a financial institution with sensitive information can be made.
- **Analysis of CVSS Scores:** The database breach scores highest because it results in broad and high-impact data and cofidentiality loss with less barriers for such attacks with very poor encryption.

Overall Critical Evaluation

Both tools, in their specific application to the recent incidents at eSecureFin, identify the breach of the database as the most serious current risk, closely followed by phishing (frequently an attack vector) and insider threat. While the risk matrix is perhaps more instructive regarding business consequences-e.g., erosion of client trust, fines from regulatory bodies-the CVSS gives standardized technical severity on which a comparison can be based. Neither captures the cascading effects, such as phishing likely leading to the database breach. Such recurrence of incidents in less than one year underlines systemic failures in both preventive and detective controls. Without urgent upgrades, eSecureFin remains very susceptible to further costly breaches in a sector where data value encourages relentless attackers.

Task 2: Develop software solutions with consideration for data security risks and secure data storage principles.

Colab Link : https://colab.research.google.com/drive/1uz2p7JWAVgoCZ7Bg53GsjADusLg42Aqz?usp=sharing

1. Fernet Symmetric Scripting

code:

DataSecurity.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Encrytion and Decryption

[12]

```python
# Sample sensitive text (like customer data in eSecureFin)
original_text = b"Customer: Jameel Ahmed | NINO: AB123456C | Credit Card: 4111-1111-1111-1111 | Balance: $5000"

# Encrypting the text
encrypted = fernet.encrypt(original_text)

print("Encrypted data:")
print(encrypted)

# Decrypting (to verify it works)
decrypted = fernet.decrypt(encrypted)

print("\nDecrypted data:")
print(decrypted.decode())  # Shows original text again
```

```
Encrypted data:
b'gAAAAABpY_VIsGVfpOUmbOaWJrXB-RAOQm-QAxXl6MCRUkPJ0ZqOkQpyBA3o0lJi9tTKi855AI5lG1syKLb6zWtja7d3a71X5n2CAJGhv5wnSsj

Decrypted data:
Customer: Jameel Ahmed | NINO: AB123456C | Credit Card: 4111-1111-1111-1111 | Balance: $5000
```

Explanation: Encrypts sensitive data of customers' data file so it become unreadable without the key. It prevents data exposure if data file is stolen. In this way data in the company will be safe if attacker tries to breach the data. This is the best ans easy way to encrypt the customers' credentials from attack and this keeps surety about company's integrity.

2. Random Password Generator

Code:

DataSecurity.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RANDOM PASSWORD GENERATOR

[13] 3s

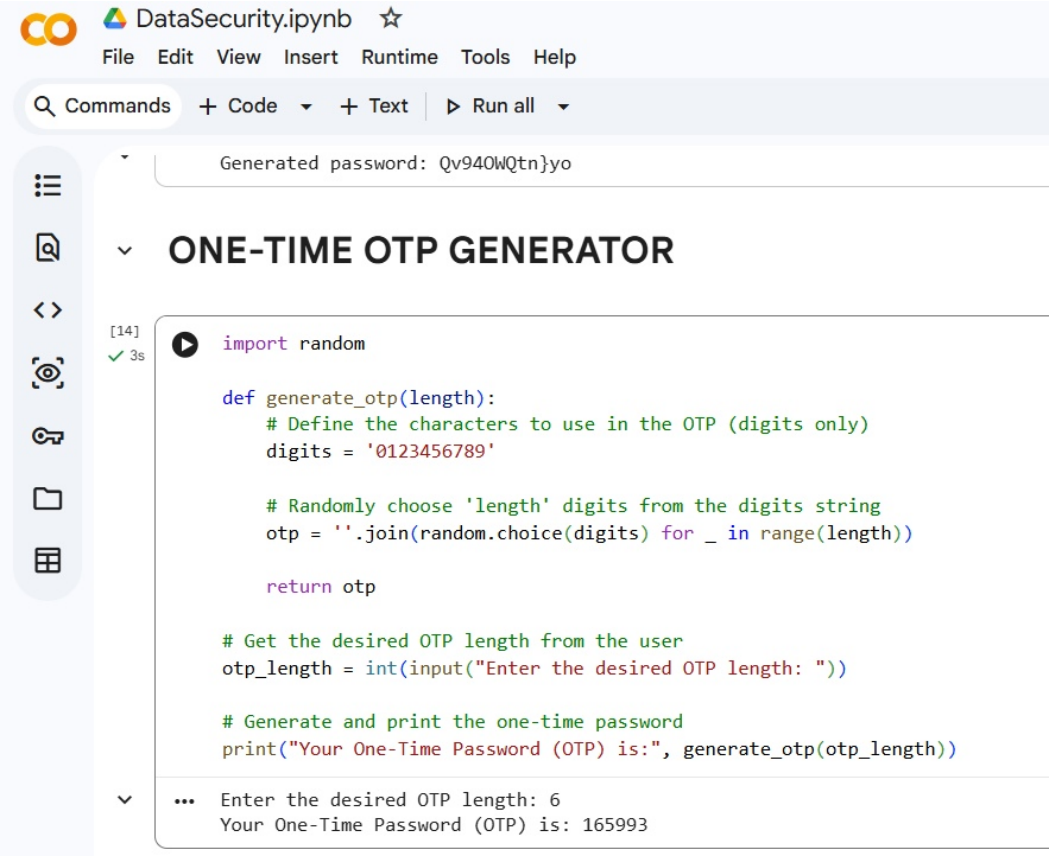
```python
import random

def generate_password(length):
    # Manually define the characters to use in the password
    letters = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'  # Uppercase and lowercase letters
    digits = '0123456789'                                              # Digits (0-9)
    symbols = '!@#$%^&*()-_=+[]{};:,.<>?/|`~'                          # Special characters

    # Combine all the characters into one string
    all_characters = letters + digits + symbols

    # Randomly choose 'length' characters from the combined string
    password = ''.join(random.choice(all_characters) for _ in range(length))

    return password

# Get the desired password length from the user
password_length = int(input("Enter the desired password length: "))

# Generate and print a random password
print("Generated password:", generate_password(password_length))
```

```
Enter the desired password length: 12
Generated password: Qv94OWQtn}yo
```

Explanation: Helps user and employees to generate random passwords using desired password length, so that they can get clue of making passwords. This will lead to generate passwords that are not easily guessable, making brute-force and phishing attacks much harder. Many of the users feel difficulty in setting a password, this will help the user to create a random password between a desired length, max length means more random password and this becomes nightmare for attackers to brute-force such a large pasword consist of digits, letter and symbols.

3. One-Time OTP Generator

Code:



Explanation: Produces short-lived one time passwords for authentication. It adds an extra layer of security against credentials replay for authentication and reduces phishing attacks. It limits the window for unauthorized logins. Passwords ensure successful authorizations but if someone's password has been stolen then OTP helps to make unsuccessful authentication of the attacker while with same password. Most of the sites used this concept to make authentications integrity.

4. Password Validator

Code:

DataSecurity.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

PASSWORD VALIDATOR

[16] 17s

```python
while True:
    password = input("Give me a password(8-12): ")

    # Check length (between 8 and 12 characters)
    if len(password) < 8 or len(password) > 12:
        print("Password must be between 8 and 12 characters. Try again.")
        continue

    # Check for at least one uppercase letter
    if not any(char.isupper() for char in password):
        print("Password should have at least one uppercase letter.")
        continue

    # Check for at least one digit
    if not any(char.isdigit() for char in password):
        print("Password should have at least one number.")
        continue

    # If all checks pass
    print("Password is OK!")
    break
```

```
Give me a password(8-12): Jameel313
Password is OK!
```

Explanation: It authenticates the password credentials of the user, reduces the chances of successful login of unauthorized credentials. Makes system more secure only for authorized persons only, limits the chances of phishing attacks and data breach. This is best way to ensure user to make password according to instructions and these types of passwords need very much time to be brute-forced. Passwords length of 12 with mix of letters, digits and symbols need centuries to be guessed. So, this is best way for keeping authentications unguessable.

5. Password Validation using python libraries

Codes:

DataSecurity.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

PASSWORD VALIDATION USING PYTHON LIBRARIES

Password_validator Library

[17]

```
!pip install password-validator
```

[20] 0s

```
from password_validator import PasswordValidator

# Create a schema
schema = PasswordValidator()

schema.min(8).max(15).has().uppercase().has().lowercase().has().digits()
schema.has().no().spaces()

print(schema.validate("Password123"))
print(schema.validate("Passwrod£$%"))
print(schema.validate("PsSWoRD123$*%"))
print(schema.validate("Passwo rd1"))
```

```
True
False
True
False
```

DataSecurity.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Regex Expression Library

[21] 0s

```python
import re

def validate_password(password):
    if len(password) < 8:
        return False
    if not re.search("[a-z]", password):
        return False
    if not re.search("[A-Z]", password):
        return False
    if not re.search("[0-9]", password):
        return False
    return True

password = "Password1"
if validate_password(password):
    print("Valid password")
else:
    print("Invalid password")
```

```
Valid password
```

Explanation: These help same as passoword validator method discussed before, the difference is that they are more secure and easy way to achieve this goal of credentials validation before successful login. It reduces the chances of any kind of attack on system or it's data. This is best way to ensure user to make password according to instructions and these types of passwords need very much time to be brute-forced. Passwords length of 12 with mix of letters, digits and symbols need centuries to be guessed. So, this is best way for keeping authentications unguessable. And all the work is handled by the python built-in libraries, no extra complexities to get such validations.

6. MFA (Time-based OTP Scripting)

Code:

DataSecurity.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[45] 0s

```python
import pyotp

pin = pyotp.TOTP('base32secret3232')
print("Your OTP is:",pin.now())
```

```
Your OTP is: 410695
```

[46] 4s

```python
verification_code = input("Please enter your verification code: ")
if pin.verify(verification_code)==True:
  print("Successful login")
else:
  print("Unsuccessfull login")
```

```
Please enter your verification code: 410695
Successful login
```

[47] 11s

```python
verification_code = input("Please enter your verification code: ")
import time
while(True):
  if pin.verify(verification_code)==True:
    print("Successful login")
  else:
    print("Unsuccessfull login")
    break
  time.sleep(1)
```

```
Please enter your verification code: 410695
Successful login
Successful login
Successful login
Successful login
Successful login
Successful login
Successful login
Unsuccessfull login
```

Explanation: This method works in multi-factor authentication as it generates time-based OTP, if user has not used OTP under 30 seconds, then he has to seek for other OTP. This helps in more security for OTPs too, reduces scams for accessing OTPs from customers. Asking OTPs from customers takes time and this fails the attemp of the attacker using customer's OTP. Adds multi-layered security in the system. So this is also best approacht to use in ensecureFin to aviod from attacks.

7. Password Hashing

Code:

DataSecurity.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

PASSWORD HASHING

[48]
```
!pip install bcrypt
```

[49]
```
import bcrypt

# example password
password = 'passwordabc'
# converting password to array of bytes
bytes = password.encode('utf-8')
# generating the salt
salt = bcrypt.gensalt()
# Hashing the password
hash = bcrypt.hashpw(bytes, salt)
# Taking user entered password
userPassword = 'password000'
# encoding user password
userBytes = userPassword.encode('utf-8')
# checking password
result = bcrypt.checkpw(userBytes, hash)

print(result)
```

```
False
```

[50]
```
import bcrypt

# example password
password = 'passwordabc'
# converting password to array of bytes
bytes = password.encode('utf-8')
# generating the salt
salt = bcrypt.gensalt()
# Hashing the password
hash = bcrypt.hashpw(bytes, salt)
# Taking user entered password
userPassword = 'passwordabc'
# encoding user password
userBytes = userPassword.encode('utf-8')
# checking password
result = bcrypt.checkpw(userBytes, hash)

print(result)
```

```
True
```

Explanation: It stores passwords securely using salted hashing. Even if the databased is accessed or breached by attackers, they can't recover usable passwords. As the passwords are not same everytime, it uses key to be decrypt, so without key they are useless if password is same, so it is impossible for attacker to use these passwords to access or breach data or successful logins. Adds and extra layer of protection in credentials theft and insider access.

Task 3: Identify and recommend decisive actions to bridge security gaps and improve security.

Identification, Description, and Evaluation of Common Cybersecurity Frameworks

Cybersecurity frameworks provide structured approaches to manage risk, in particular for financial firms such as eSecureFin, in which the impact of data breaches can be devastating. I considered four recommended ones:

- **NIST Cybersecurity Framework (CSF):** It is developed by the National Institute of Standards and Technology (NIST, Feb 2014), this framework has five core functions: Identify, Protect, Detect, Respond, and Recover. It is flexible and free, making it available for all types of oranisations. **Evaluation:** Strong and comprehensive framework, however, it requires customisation as it lacks some measurement metrics.
- **ISO 27001:** International Organization for Standardization standard that provides requirements for the organization, implementation, and maintenance of an ISMS. It includes the assessment and treatment of information security risks, the implementation of the controls concerning confidentiality/integrity/availability, and certification audits. **Evaluation:** Very good, as it means formal compliance and international recognition, but the implementation and certification are very costly and may overburden a growing firm like eSecureFin.
- **CIS Controls:** This has come from the Center of Internet Security and provides a prioritized list of 18 agreed-upon actions (inventory and assign assets, secure configurations, protection of data) and so on (CIS, 2021). **Evaluation:** CIS is very useful in terms of immediate actions and costs; however, it's more tactical compared to NIST.
- **COBIT:** This IT governance and management framework, produced by ISACA (WIKI, 2019), promotes IT security and its alignment with enterprise strategic objectives in risk optimization and resource management processes. **Evaluation:** Although this methodology has its uses in IT security and strategic corporate plans, it would be overkill in its complexity for use in a company like eSecureFin's current structure and size.

Recommended Framework

I would like to recommend that the NIST CSF be adopted for eSecureFin. This framework would serve as an effective pathway in dealing with the recent events by grouping responses under the five functions.

Rationale for Choosing NIST CSF for the Scenario

The challenges faced in eSecureFin, such as phishing to steal credentials, internal data theft, and the breach of the database, arise from the lack of protection and surveillance. NIST CSF is the best because:

- It is free and highly adaptable and requires minimal financial costs even for a rapidly growing organization.
- The "Identify" feature assists with asset and risk identification, such as vulnerable database identification.
- "Protect" specifically addresses issues concerning the lack of access controls and encryption (such as the use of multi-factor authentication to mitigate phishing):
- "Detect" enhances insider threat detection.
- "Respond" and "Recover" enable speedy action in case of a breach as seen in the database leak, thus minimizing downtime.
- It complies with UK regulation (for example, FCA rules) and can be coupled with software from Task 2, such as encryption scripts to protect data. Unlike the certification focus of ISO or even the tactical list of CIS, NIST provides strategic information that is relevant to risks involving financial data.

Linking to Task 2 Tools for Security Improvements

The Python scripts of Task 2 can be combined into NIST's "Protect" function. As an example, the Bcrypt hashing script-screenshot below-suggests storing passwords securely to prevent any credential reuse in a phishing attack. This would actually be the implementation of the weak access control gap.

DataSecurity.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

PASSWORD HASHING

[48]

```
!pip install bcrypt
```

[49]

```
import bcrypt

# example password
password = 'passwordabc'
# converting password to array of bytes
bytes = password.encode('utf-8')
# generating the salt
salt = bcrypt.gensalt()
# Hashing the password
hash = bcrypt.hashpw(bytes, salt)
# Taking user entered password
userPassword = 'password000'
# encoding user password
userBytes = userPassword.encode('utf-8')
# checking password
result = bcrypt.checkpw(userBytes, hash)

print(result)
```

```
False
```

[50]

```
import bcrypt

# example password
password = 'passwordabc'
# converting password to array of bytes
bytes = password.encode('utf-8')
# generating the salt
salt = bcrypt.gensalt()
# Hashing the password
hash = bcrypt.hashpw(bytes, salt)
# Taking user entered password
userPassword = 'passwordabc'
# encoding user password
userBytes = userPassword.encode('utf-8')
# checking password
result = bcrypt.checkpw(userBytes, hash)

print(result)
```

```
True
```

This tool ensures passwords remain confidential even if breached, directly improving security post-incidents.

TASK 4: SUMMARY REPORT

This portfolio has been a practical learning process for data security for eSecureFin Limited, which is a growing financial services business that has been hit with three significant events over the past year, including a phishing scam that attacked employee login credentials, an insider threat that saw a former employee steal data, and an unauthorised database breach that saw thousands of customers' personal information breached.

Task 1 – Risk Assessment

The first exercise taught me to assess the level of security risks with a critical approach by combining two methods: the risk matrix and CVSS rating. Each method has been used to assess the recent events at eSecureFin's system, and the results clearly indicated that the threat level was higher due to the database theft scenario with an extreme level of risk and high CVSS rating; phishing attack was the second most critical threat due to its high entry point, followed by an internal threat. This exercise taught me to assess risks considering both subjective and objective factors for prioritizing actions. It pointed to the way in which poor controls, such as inadequate training, weak encryption methods, and the absence of surveillance, contributed to these events occurring and might facilitate further such occurrences.

Task 2 - Security Scripts in Python

For Task 2, I identified and showcased seven different Python scripts that I learned in class this semester and relate to protecting some of these weaknesses that exist in this company. Some examples that I included were password hashing using bcrypt to protect against theft, encryption using Fernet to protect stored information, two-factor authentication using TOTP to prevent a phishing attack from succeeding, hashing files to ensure that they haven't been modified, password generation to promote using strong passwords, OTP generation as an added layer of protection during login attempts, and password strength checks to promote best practices. The important concept learned here is that often simple and specialized code is all that is required to improve or fix weaknesses within the CIA triad.

Task 3: Recommending Framework

Task 3 was an eye-opener for recognizing the importance of structured frameworks. Having evaluated NIST Cyber Security Framework, ISO 27001, CIS Controls, and COBIT, I identified that NIST Cyber Security Framework is ideal for my growing business eSecureFin because of its flexibility, scalability, and minimal costs (comparatively), offering all three tasks' solutions in five functions (short for tasks here: Identify, Protect, Detect, Respond, Recover). Corresponding NIST's Protect function for scripts in Task 2 (for example, bcrypt hashing) was an interesting experience.

Overall Key Learnings

This portfolio has enhanced my knowledge about the intersection between theory and practice as it applies to data security. The cases within eSecureFin did not occur because of one failure, but instead a variety of small failures lack of training, weak passwords, no encryption, lack of monitoring. The key takeaway is that data security is executed through multiple means passwords, encryption, 2FA, monitoring, and risk management. It is also important to focus on risk specifically, rather than generally, as well as leverage risk matrices, CVSS, or Python scripts to outline potential improvements. Perhaps the most important factor, though, is that data security is constantly changing.

This exercise has given me the confidence to approach my future career as a data security consultant. With this exercise, I have gained the knowledge to apply risk analysis, use coding practices, and provide frameworks on how to secure financial data to restore trust after a data breach.

Reference List

1. Center for Internet Security (CIS) (2021) *18 is the new 20: CIS Controls v8 is here*. Available at: https://www.cisecurity.org/insights/blog/18-is-the-new-20-cis-controls-v8-is-here (Accessed: 12 January 2026).
2. National Institute of Standards and Technology (NIST) (2014) *Framework for Improving Critical Infrastructure Cybersecurity, Version 1.0*. Available at: https://www.nist.gov/cyberframework/history-and-creation-framework (Accessed: 12 January 2026).
3. Wikipedia (2019) *COBIT*. Available at: https://en.wikipedia.org/wiki/COBIT (Accessed: 12 January 2026).

Appendix

Aerial photo of students collaborating at a shared desk - 1197x282

Appendix item: Aerial photo of students collaborating at a shared desk - 1197x282

Show location

Aerial photo of students collaborating at a shared desk - 1197x282

png (656 KB)